



KILN

DeFi Integrations Security Assessment Report

Version: 2.0

December, 2024

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	3
Detailed Findings	5
Summary of Findings	6
Deposit() and Withdraw() Vault Functions Frequently Revert	7
EVM PUSH0 Opcode Not Supported On Certain Layer 2s	9
Non-standard ERC-20 Token Vaults Cannot be Upgraded	10
Incorrect Accounting For Additional Rewards	11
Unbounded FeeRecipient Loop Lengths	12
Bad Debt Recovery Will Be Charged Performance Fee	13
ERC-4626 Specification Deviation	14
Claim Manager Can Control Additional Rewards	15
Repeated Reward Claim Logic	16
Miscellaneous General Comments	17
A Test Suite	19
B Vulnerability Severity Classification	20

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Kiln smart contracts. The review focused solely on the security aspects of the Solidity implementation of the contract, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the smart contract. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Kiln smart contracts contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

Outputs of automated testing that were developed during this assessment are also included for reference (in the Appendix: [Test Suite](#)).

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Kiln smart contracts.

Overview

Kiln DeFi is a protocol that allows integrators to run `ERC-4626` vaults on behalf of their end users.

These vaults interact with various DeFi protocols on numerous different chains to generate income for users. This includes popular platforms such as Aave and Compound on Ethereum mainnet, as well as forks and other unique projects on various Layer 2 networks.

The vaults are designed to allow integrators to have a degree of freedom over what fees to charge for management and performance, while also being able to adhere to any legal restrictions inherent to their jurisdiction.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [defi-integrations repository](#).

The scope of this time-boxed review was strictly limited to files at commit [85bff44](#). The fixes of the identified issues were assessed at commit [d591799](#).

Note: third party libraries and dependencies, such as OpenZeppelin, were excluded from the scope of this assessment.

Approach

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

For a more detailed, but non-exhaustive list of examined vectors, see [\[1, 2\]](#).

To support this review, the testing team also utilised the following automated testing tools:

- Mythril: <https://github.com/ConsenSys/mythril>
- Slither: <https://github.com/trailofbits/slither>
- Surya: <https://github.com/ConsenSys/surya>
- Aderyn: <https://github.com/Cyfrin/aderyn>

Output for these automated tools is available upon request.

Coverage Limitations

Due to a time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 10 issues during this assessment. Categorised by their severity:

- High: 3 issues.

- Medium: 1 issue.
- Low: 1 issue.
- Informational: 5 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Kiln smart contracts. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the contracts, including gas optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open:** the issue has not been addressed by the project team.
- **Resolved:** the issue was acknowledged by the project team and updates to the affected contract(s) have been made to mitigate the related risk.
- **Closed:** the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
KLN2-01	Deposit() and Withdraw() Vault Functions Frequently Revert	High	Resolved
KLN2-02	EVM PUSH0 Opcode Not Supported On Certain Layer 2s	High	Resolved
KLN2-03	Non-standard ERC-20 Token Vaults Cannot be Upgraded	High	Resolved
KLN2-04	Incorrect Accounting For Additional Rewards	Medium	Closed
KLN2-05	Unbounded FeeRecipient Loop Lengths	Low	Closed
KLN2-06	Bad Debt Recovery Will Be Charged Performance Fee	Informational	Closed
KLN2-07	ERC-4626 Specification Deviation	Informational	Closed
KLN2-08	Claim Manager Can Control Additional Rewards	Informational	Closed
KLN2-09	Repeated Reward Claim Logic	Informational	Resolved
KLN2-10	Miscellaneous General Comments	Informational	Resolved

KLN2-01	Deposit() and Withdraw() Vault Functions Frequently Revert		
Asset	Vault.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

Calls to `deposit()` and `withdraw()` can frequently revert due to an interaction between the Vault `ERC-4626` share system and additional assets gained from standard connector activities. This can interfere with end users attempting to withdraw or deposit to the related vault.

While users can still exit or enter positions using the *share-exact* `mint()` or `redeem()` functions, it is more likely end users would like to specify their actions in the underlying asset via `deposit()` or `withdraw()`, meaning this would delay most user interactions.

This occurs due to the `_checkPartialShares()` function that is called on all user interactions to prevent users receiving small quantities of shares that may not be economic to interact with. When a connector gains additional assets, usually through interest accrued in the underlying investment platform, it will mint new shares via `_accrueRewardFee()` to represent this.

As these new shares are minted prior to calling `_checkPartialShares()`, it is possible for them to change the ratio of assets to shares in the system and so lead to the quantity of assets a user wishes to deposit or withdraw to generate a quantity of shares that includes an undesirable small remainder amount. This will result in the overall call reverting, blocking the intended action.

Recommendations

Several strategies can be applied to mitigate this issue:

- Remove the partial shares check, although this may leave users with small quantities of shares that are not economic to remove or interact with;
- Round down partial share remainders when they occur;
- If no business logic changes are desired, then the vault could be used so long as all UIs point only to *share-exact* `mint()` and `redeem()` functions.

Resolution

The development team has fixed this issue in [PR#221](#) by rounding down partial share remainders when they occur. To do so, the development team has implemented a new function `_roundDownPartialShares()` which round down the shares as follow:


```
function _roundDownPartialShares(uint256 shares) internal view returns (uint256) {
    uint8 _offset = _decimalsOffset();
    if (_offset > 0) {
        shares -= shares % 10 ** _offset;
    }
    return shares;
}
```

The changes have been applied to the functions `previewDeposit()`, `previewWithdraw()` and `withdraw()`.

KLN2-02 EVM <code>PUSH0</code> Opcode Not Supported On Certain Layer 2s			
Asset	/*		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

Some Layer 2 networks do not support certain Opcodes and are not as up-to-date as the Ethereum mainnet's version of the EVM. In particular, several Layer 2 networks do not support the `PUSH0` opcode introduced in version `0.8.20` with the Shanghai update. Solidity code compiled with a `pragma` equal to or higher than `0.8.20` and EVM version Shanghai or newer will fail to deploy to unsupported networks.

This will make contracts unavailable on certain networks. In particular, the currently affected networks were:

- Linea
- Flare Network
- ZkSync

This list is not exhaustive and may grow as Kiln deploys to more chains.

Recommendations

Adding EVM version `London` to the compiler settings or making use of Solidity version `0.8.19` will allow contracts to be deployed on related Layer 2 networks.

For the reason of preserving testing outcomes, restricting the EVM version is more advisable, as Foundry does not ensure the test EVM matches the onchain EVM version when performing fork tests.

Resolution

The development team has confirmed that they will use the `London` EVM version when deploying the project to the mentioned chains.

KLN2-03	Non-standard ERC-20 Token Vaults Cannot be Upgraded		
Asset	Vault.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

A vault with an ERC-20 token which does not strictly follow the ERC-20 token interface cannot be upgraded.

This is because the token's `approve()` function does not adhere to the ERC-20 interface. One prominent example for such tokens is the stablecoin USDT which is supported by the Kiln Protocol.

When trying to upgrade the vault to the new implementation using the function `upgrade()`, the transaction reverts for two reasons:

1. Certain tokens do not have a `bool` return value for `approve()` which is expected in the interface `IERC20`. The result is that the call `IERC20(asset()).approve()` in `__Vault_upgrade()` on line [412] will revert due to an incorrect return value.
For example in the case of USDT, the function does not return any value, which causes an execution error as a `bool` is required.
2. Certain tokens require the allowance to be set to zero before they can be set to a non-zero value. Again USDT is an example of this, causing a revert to occur as the previous allowance is a non-zero value.

Recommendations

Use `forceApprove()` from the OpenZeppelin library `SafeERC20` instead of the default `approve()` method.

Resolution

This issue has been fixed in [PR#219](#) as recommended. The function `approve()` is replaced by the function `forceApprove()` to avoid the revert of the non-standard ERC20 tokens.

KLN2-04 Incorrect Accounting For Additional Rewards			
Asset	Vault.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Some vaults have the ability to claim additional rewards. However, as these rewards can only be claimed by the holder of the `CLAIM_MANAGER_ROLE`, this will lead to rewards being allocated to the wrong users. If a user withdraws from the vault prior to a call to `claimAdditionalRewards()`, they will lose any additional rewards their deposit had earned while in the vault. Likewise, if a user front runs a call to `claimAdditionalRewards()`, it is possible for them to claim additional rewards relating to a deposit period prior to their entry to the vault.

Recommendations

Additional rewards should be accounted for using a parallel accounting system that tracks user deposit and withdrawal times, and attributes them shares accordingly. This process would be a complex endeavour; however so some alternative solutions are:

- Making use of the claim strategy that allows additional rewards to be sent to the fee recipients address which could then implement additional claim logic based on historic onchain data.
- Adding a delayed reward streaming system to additional rewards that can allocate them over a period of time such as a week or a month. While the attribution of this method could still be skewed, it would help prevent frontrunning calls and increase the predictability of such rewards.

Resolution

The development team has acknowledged this issue, with the intentions to use a parallel solution for the reward distribution.

KLN2-05	Unbounded FeeRecipient Loop Lengths		
Asset	FeeDispatcher.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The length of the `_feeRecipients` is unbounded: this can cause various functions that iterate through all entries of `_feeRecipients` to reach the block gas limit and revert in extreme cases. While this limit is very high for Ethereum mainnet, it is possible that for some Layer 2s the block gas limit or transaction gas limit may be much lower.

Issues can arise when updating vaults, setting new `_feeRecipient` arrays using `setFeeRecipients()`, or dispatching accrued fees via `dispatchFees()`.

Recommendations

It is recommended the Kiln team make vault integrators aware of this limitation and document the effective max length of `_feeRecipients` on chains vaults are deployed on. Alternatively, an artificial safe cap could be added to `_feeRecipients` to ensure this issue does not arise after vault setup.

Resolution

This issue has been acknowledged.

KLN2-06	Bad Debt Recovery Will Be Charged Performance Fee	
Asset	Vault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

In the event a connector experiences a loss; which, after some time, is recovered, the vault depositors will be charged a performance fee on this recovery amount despite it not being a performance profit.

This occurs because in the rare occasion that a connector's total assets decreases, the vault will reduce the total assets recorded. If this happens and then the lost funds are recovered through insurance or other legal means, the vault's total assets will rise again and the next `_accrueRewardFee()` will allocate the `_rewardFee` to the fee collector.

The result is undesirable for the vault depositors as these amounts may be either the original deposit assets or rewards that have already been charged a reward fee.

It is noted that this scenario is likely to be rare as not all connectors are capable of realising a loss in this manner and even those that are would need to replenish said loss due to asset recovery or insurance payout.

Recommendations

Due to the unlikelihood of this scenario occurring, the best solution may be for Kiln to monitor connectors for losses and reach out to coordinate with the underlying protocols team to ensure any recovered losses are accounted for correctly, or temporarily setting the `rewardFee` to zero in cases where losses are detected.

Resolution

This issue has been acknowledged by the developing team as it is documented and a known issue. Therefore, the testing has reduced the severity of this issue to *informational*.

KLN2-07	ERC-4626 Specification Deviation	
Asset	Contract.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The underlying asset of an ERC-4626 vault is intended to be an ERC-20 specification compliant token. However, several existing vaults have underlying tokens that are not fully ERC-20 compliant, such as USDT. While a specific issue was noted for upgrading USDT vaults, other non-ERC-20 tokens may pose different integration risks as ERC-4626 vaults are not intended to be used with them.

Recommendations

Vault integrators should be advised to ensure the underlying token their vault uses is ERC-20 compliant. In the event a non-ERC-20 token is required, it should be thoroughly assessed to ensure it will integrate correctly with all vault contracts.

Resolution

This issue has been acknowledged by the development team.

KLN2-08	Claim Manager Can Control Additional Rewards	
Asset	Vault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

If the additional reward strategy for a vault is set to `reinvest` or `claim`, then it is possible for the claim manager to send these additional reward tokens to any address. While the claim manager is a trusted role, it is worth highlighting this risk in case the controlling party is compromised.

Recommendations

Ensure the claim manager role is sufficiently protected by making use of a multisig.

Additionally, consider adding logic that provides validation to transfer recipients and swap payloads.

Resolution

The development team has acknowledged this issue.

KLN2-09	Repeated Reward Claim Logic	
Asset	KineticConnector.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

The Kinetic connector contains logic to claim `wFLR` tokens that will repeat claims redundantly, leading to waste of gas. When calling `rFLR.claimRewards()`, each call will iterate through all eligible claims since the specified month until the current month. This makes calling for each month, the current design, unnecessary.

Furthermore, every time `claim()` is called for `wFLR` rewards we always claim since the start month of the distribution even if there has been a call before.

Recommendations

Allow the calls to `claim()` to store the month to process claims from in the `payload` input or add additional logic to record the current month and previously claimed months in the `KineticConnector` contract.

Resolution

This issue has been fixed in [PR#220](#) by removing the loop that iterates through all months since deployment.

KLN2-10	Miscellaneous General Comments	
Asset	All contracts	
Status	Resolved: See Resolution	
Rating	Informational	

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Single Swap Target May Be Insufficient

*Related Asset(s): connectors/**

Each connector makes use of an immutable `swapTarget` address that is the destination for swaps designed to reinvest additional rewards earned by the vault. Only one such address is allowed per connector and this may be insufficient if multiple reward tokens are claimed from a connector or if swap liquidity moves from one dApp to another.

Plan ahead for connectors where multiple additional reward tokens are possible by making use of swap aggregation services such as DEX aggregators, or use a self-deployed endpoint proxy so that the swap route can be altered in the future.

2. Inaccurate Comments and Typos

Several inaccuracies and typos were noted in reviewing the codebase:

- On line [221] of `Vault.sol`, "addtionnal" should read "additional"
- On line [231] of `Vault.sol`, the comment deviates from the protocol specification as it suggests you can skip sanctions being used by not setting a `sanctionList` address. The specifications suggest the Chainalysis sanction list is always in effect. The correct approach should be reflected in both areas to prevent confusion.
- On line [195] of `FeeDispatcher.sol`, there is a misleading comment, fee splits will sum in `underlyingDecimal` scaling which is not always 18 digits.
- On line [193] of `AaveV3Connector.sol`, the comment incorrectly mentions COMP when the connector can have multiple different reward assets.
- On line [24] of `SUSDSConnector.sol`, the comment incorrectly mentions sDAI which was the legacy token for sUSDS.
- On line [38] of `AaveV3Connector.sol`, the interface should be named `IPoolDataProvider` not `PoolDataProvider`, to match interface naming standards.

Typos and comment inaccuracies should be fixed to improve readability.

3. Magic Numbers

Related Asset(s): VaultFactory.sol

Several values are hardcoded throughout the body of the code which can make it difficult to update or recognise their purpose.

Adding hardcoded values as named constants declared at the beginning of contracts improves readability and maintainability.

4. Unclear Pause Timestamps

Related Asset(s): *ConnectorRegistry.sol*

The `pauseTimestamp` and `Paused` events make use of the type max value such as `type(uint256).max` to denote a paused connector. This timestamp is supposed to be the time when a connector is unpaused but with such large values these times will never occur.

It is suggested to make their meaning clearer by replacing them with a boolean paused status.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows.

1. Issue acknowledged.
2. Typos and comments has been fixed it [PR#226](#).
3. Issue acknowledged.
4. Issue acknowledged.

Appendix A Test Suite

A non-exhaustive list of tests were constructed to aid this security review and are given along with this document. The `forge` framework was used to perform these tests and the output is given below.

```
Ran 7 tests for test/tests-fork/CompoundV3Connector.t.sol:CompoundV3ConnectorTest
[PASS] test_comp_claim() (gas: 152387)
[PASS] test_constructor() (gas: 20808)
[PASS] test_deposit() (gas: 494976)
[FAIL. Reason: Bob should not receive any rewards: 2261492329 != 124999999] test_frontrunRewards_Vuln() (gas: 1435131)
[PASS] test_reinvest() (gas: 828202)
[FAIL. Reason: revert: User Alice should have shares relating to rewards] test_unclaimedRewardsLost_Vuln() (gas: 1572042)
[PASS] test_withdraw() (gas: 599771)
Suite result: FAILED. 5 passed; 2 failed; 0 skipped; finished in 523.81ms (53.91ms CPU time)

Ran 3 tests for test/tests-fork/AaveV3Connector.t.sol:AaveV3ConnectorTest
[PASS] test_constructor() (gas: 20858)
[PASS] test_deposit() (gas: 707656)
[PASS] test_withdraw() (gas: 819972)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 529.42ms (12.01ms CPU time)

Ran 4 tests for test/tests-fork/FeeDispatcher.t.sol:FeeDispatcherTest
[PASS] test_dispatchFees() (gas: 473711)
[PASS] test_dispatchFees_GasLimit() (gas: 12598162)
[PASS] test_setFeeRecipients() (gas: 187455)
[PASS] test_setFeeRecipients_SameRecipient() (gas: 24130)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 530.46ms (25.51ms CPU time)

Ran 12 tests for test/tests-fork/Vault.t.sol:VaultTest
[PASS] test_constructor() (gas: 19268)
[PASS] test_deposit_Vault() (gas: 571838)
[PASS] test_deposit_existing_usdc_vault() (gas: 813477)
[PASS] test_forceWithdraw() (gas: 1281282)
[PASS] test_mint_Vault() (gas: 574343)
[PASS] test_mint_existing_usdc_vault() (gas: 837761)
[PASS] test_redeem_vault() (gas: 579289)
[FAIL. Reason: revert: reward fees should not accumulate] test_rewardFeeAppliedTwice_Vuln() (gas: 1021383)
[PASS] test_verifyUpgrade_usdcVault() (gas: 497439)
[PASS] test_withdrawMax_vault() (gas: 640709)
[PASS] test_withdraw_existingVault() (gas: 904192)
[PASS] test_withdraw_vault() (gas: 678208)
Suite result: FAILED. 11 passed; 1 failed; 0 skipped; finished in 534.93ms (102.48ms CPU time)

Ran 2 tests for test/tests-fork-arb/FeeDispatcher.t.sol:FeeDispatcherTest
[PASS] test_dispatchFees() (gas: 472586)
[PASS] test_setFeeRecipients() (gas: 187455)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 680.39ms (4.10ms CPU time)

Ran 11 tests for test/tests-fork-arb/VaultArb.t.sol:VaultArbTest
[PASS] test_constructor() (gas: 19268)
[PASS] test_deposit_Vault() (gas: 607951)
[PASS] test_deposit_existing_usdc_vault() (gas: 825913)
[PASS] test_forceWithdraw() (gas: 1299528)
[PASS] test_mint_Vault() (gas: 610346)
[PASS] test_mint_existing_usdc_vault() (gas: 850152)
[PASS] test_redeem_vault() (gas: 614512)
[PASS] test_verifyUpgrade_usdcVault() (gas: 497343)
[PASS] test_withdrawMax_vault() (gas: 678957)
[PASS] test_withdraw_existingVault() (gas: 919098)
[PASS] test_withdraw_vault() (gas: 716560)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 699.85ms (105.80ms CPU time)

Ran 1 test for test/tests-fork/VaultFactory.t.sol:vaultFactory
[FAIL. Reason: EvmError: Revert] test_upgradeVault_Vuln() (gas: 1040611218)
Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 5.36s (4.85s CPU time)

Ran 7 test suites in 5.36s (8.86s CPU time): 36 tests passed, 4 failed, 0 skipped (40 total tests)
```

Appendix B Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact				
High		Medium	High	Critical
Medium		Low	Medium	High
Low		Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

References

- [1] Sigma Prime. Solidity Security. Blog, 2018, Available: <https://blog.sigmaprime.io/solidity-security.html>. [Accessed 2018].
- [2] NCC Group. DASP - Top 10. Website, 2018, Available: <http://www.dasp.co/>. [Accessed 2018].

σ'